

Semana 4: implementação e teste isolado da Flow Identification Unit

Projeto: CI Digital Inatel, T25S2, Grupo 2

Responsável pela frente: Bruno

Estado em 9 de outubro de 2026: primeira versão funcional compilada e simulada isoladamente no ModelSim; integração com Packet Parser e Hash Unit ainda não realizada.

Este guia apresenta o que foi implementado, como reproduzir a simulação e o alcance do resultado. A FIU recebe os metadados publicados pelo Parser e entrega à Hash Unit a chave do fluxo, o comprimento e o timestamp do mesmo pacote.

1. Arquivos desta entrega

Arquivo	Finalidade
rtl/flow_identification_unit.sv	Implementação da FIU em SystemVerilog. Testbench isolado com comparações automáticas.
tb/tb_flow_identification_unit.sv	
validation/resultados.md	Registro do resultado observado no ModelSim e seus limites.
README.md	Comandos rápidos e premissas de integração.
Este guia em .md e .pdf	Descrição da implementação e dos testes para a equipe.

2. Contrato utilizado

A referência é a proposta **Contrato de Interface - Flow Identification Unit, versão 0.3, de 6 de outubro de 2026**, revisada sem FIFO. O documento se declara proposta pendente de alinhamento entre equipes. O módulo utiliza clock comum e reset síncrono ativo baixo (reset_n).

Origem	Campos recebidos
Packet Parser	src_ip_in[31:0], dst_ip_in[31:0], src_port_in[15:0], dst_port_in[15:0], protocol_in[7:0], packet_length_in[15:0], timestamp_in[79:0], in_valid.
Hash Unit	out_ready.

A saída à Hash Unit é flow_key_out[103:0], packet_length_out[15:0], timestamp_out[79:0] e out_valid. A FIU não calcula CRC, não consulta a Flow Table e não interpreta o timestamp ou o comprimento.

3. Formação da chave e associação dos dados

A chave unidirecional é formada sem troca de origem e destino e sem inversão de bytes:

```
flow_key_out <= {src_ip_in, dst_ip_in,  
                src_port_in, dst_port_in, protocol_in};
```

A distribuição dos campos é: IP de origem [103:72]; IP de destino [71:40]; porta de origem [39:24]; porta de destino [23:8]; protocolo [7:0].

Exemplo dirigido: 192.168.1.10:50000 -> 192.168.1.20:443, TCP (6), produz 104'hC0A8010AC0A80114C35001E
O comprimento e o timestamp são capturados na mesma borda que os cinco campos e ficam armazenados com a chave. A mesma chave pode aparecer em vários pacotes com comprimentos e timestamps diferentes; cada publicação válida continua sendo um registro distinto.

4. Comportamento temporal do RTL

1. Com `reset_n = 0` na borda de subida, a FIU cancela o registro pendente e desativa `out_valid`.
2. Em uma borda com `in_valid = 1` e saída livre, a FIU captura os 200 bits de dados. A saída registrada fica válida no ciclo seguinte; não há entrega direta na borda de captura.
3. A Hash Unit aceita o registro em uma borda com `out_valid && out_ready`. Enquanto `out_valid = 1` e `out_ready = 0`, chave, comprimento, timestamp e validade permanecem estáveis.
4. Na mesma borda em que o registro anterior é aceito, um novo `in_valid = 1` pode substituí-lo para apresentação no ciclo seguinte.
5. Sem nova publicação após uma entrega, `out_valid` cai. Os campos de dados com `out_valid = 0` não têm significado para o consumidor.

A entrada do Parser **não tem `in_ready`** nesta proposta. Portanto, uma publicação enquanto `out_valid = 1` e `out_ready = 0` viola a premissa temporal: o registro antigo é preservado, e não há espaço para guardar o novo. O RTL emite `$error` nessa situação durante a simulação. A ausência de FIFO depende da folga entre publicações e do bloqueio máximo do caminho até o Controller; essa condição ainda precisa ser conferida na integração.

5. Testes isolados realizados

O testbench aplica entradas controladas e compara automaticamente as saídas. Ele cobre:

Cenário	Critério conferido
TCP de referência	Chave exata C0A8010AC0A80114C35001BB06, comprimento e timestamp associados.
Espera da Hash Unit	Registro e <code>out_valid</code> estáveis durante três bordas com <code>out_ready = 0</code> .
Entrada inválida	Alterar entradas com <code>in_valid = 0</code> não modifica o registro pendente.
Entrega e nova publicação	A Hash Unit recebe o registro antigo e a FIU apresenta o seguinte após a mesma borda.
Sentido inverso	Origem e destino invertidos produzem chave distinta.
Mesma chave, outros metadados	A chave se repete, e o novo comprimento/timestamp são preservados.
Reset com registro pendente	A validade cai e o registro cancelado não reaparece.
UDP	Chave 0A0000001080808081234003511, comprimento 28 e timestamp conhecido.

O testbench confirma três registros efetivamente entregues. Um registro adicional é cancelado intencionalmente pelo reset. O teste não injeta uma publicação durante bloqueio porque essa condição é inválida no contrato; a checagem `$error` existe no RTL para denunciá-la se ocorrer.

6. Reprodução no ModelSim

Extraia o pacote. No **Transcript** do ModelSim, entre na pasta `semana4_fiu` e execute:

```
vlib work
vlog -sv rtl/flow_identification_unit.sv tb/tb_flow_identification_unit.sv
vsim work.tb_flow_identification_unit
run -all
```

Se a biblioteca `work` já existir, `vlib work` pode ser omitido. O teste termina com:

```
PASS: FIU, 3 registros entregues em ordem e sem perda
** Note: $finish
```

Na execução apresentada por Bruno em 9 de outubro de 2026, os dois arquivos compilaram sem erros, o PASS apareceu no Transcript e o \$finish ocorreu em **126 ns**. A captura do Transcript também mostra “Break in Module” depois de \$finish e ao interagir com a janela Wave; isso não indica uma divergência nas comparações automáticas. O registro observável não constitui avaliação de síntese, timing físico ou integração.

7. Próxima etapa de integração

Com Herman, conferir nomes e larguras reais das portas do Parser, a representação numérica dos campos e a publicação de um registro por borda com metadata_valid. Com Samuel, conferir Hash.in_ready -> FI.out_ready, a largura de 80 bits do timestamp e a aceitação do conjunto chave/comprimento/timestamp. Entre as equipes, confirmar unidade e formato do timestamp, significado de packet_length, clock/reset comuns e término do PCAP após esvaziar os registros pendentes.

A verificação integrada deverá medir ou limitar o intervalo mínimo entre publicações e o tempo máximo de bloqueio da Hash Unit e do Controller. Se a premissa temporal falhar, a interface precisará ser revista; o teste isolado não demonstra essa folga.

Conclusão

A primeira versão da FIU atende às funções previstas para a Semana 4 no **teste isolado**: recebe os metadados, forma a chave de 104 bits, preserva comprimento e timestamp de 80 bits e entrega cada registro válido por handshake. O Transcript do ModelSim confirma três entregas em ordem e sem divergência nos cenários dirigidos. A conexão aos módulos dos colegas e a validação da operação sem FIFO permanecem para a etapa seguinte.